
Detecting and Splitting Quran Recitation in Noisy ASR Transcripts: A Competition Proposal

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Before an application can tell a learner what they recited wrongly, it has to work
2 out what they recited at all: which ayahs appear in a noisy ASR transcript, where
3 each one begins and ends, and whether the audio was Quran. We propose a
4 competition on that task and release the benchmark it needs: 258 human-reviewed
5 transcripts of production recordings, 1,267 gold ayah chunks over 604 ayahs, a
6 deterministic recording-level split whose oracle floor is exactly zero, a fixed sub-
7 second scorecard, a public 26-recording review sample, checksums, and three
8 evaluated baselines spanning an order of magnitude, one of them the hand-built
9 pipeline of a deployed application.

10 1 The task and why it is open

11 Quran-memorization applications transcribe recitation and must identify ayahs and word boundaries
12 despite opening formulas, repeats, restarts, corrections and ASR errors. A learner can skip an ayah
13 or return to an earlier one. The task supports downstream feedback by recovering which reference
14 passages the transcript represents.

15 A system returns the set of recited ayah ids and a per-word assignment, or abstains on non-Quran
16 input. The benchmark focuses on noisy, unconstrained memorization attempts rather than known-text
17 transcription. A companion submission studies the interpretation of differences after ayahs have been
18 identified.

19 2 The benchmark

20 **Source and consent.** The 258 recordings come from adult users of a Quran-memorization chat bot,
21 under terms of service that disclose review for service improvement. A commercial ASR model
22 produced the transcripts, and a human reviewed every row after LLM pre-labeling, assigning ayah
23 ids, splitting the transcript and recording a confidence. 254 rows are Quranic, covering 604 distinct
24 ayahs from 45 surahs, with 1–46 ayahs and 2–261 words per recording (median 40); the remaining
25 four are non-Quranic negatives on which a system must abstain. We release text only, replace learner
26 and recording identifiers with surrogate ids, and checked the negatives by hand. Since the recited text
27 comes from a closed public canon, the transcripts carry essentially no personal information. Licence
28 CC BY 4.0.

29 **Split.** A deterministic stratified 60/40 split by *recording*, never by chunk: 151/107 recordings and
30 711/556 chunks. Splitting by chunk would leak one reciter’s ASR quirks across the boundary. The
31 negatives split 2/2 so abstention is testable held out, and the two rows whose gold assignment spans

32 surahs are pinned to train, which makes the held-out oracle floor exactly 0: feeding the gold split
 33 back scores zero, so the dataset is internally consistent and the metric has a known bottom.

34 **Sample and integrity.** A 26-recording sample, ten percent of the set, is published for review. It
 35 carries both negatives, a non-contiguous recitation, a medium-confidence row, a long al-Baqarah
 36 range and a recording of over 120 words, so a reviewer sees every behaviour the metric scores. Every
 37 released file is pinned by SHA-256.

38 3 Metric and baselines

39 The scorecard is a fixed script; lower is better; each of the three components lies in $[0, 1]$ and is printed
 40 beside the scalar: $\text{research_score} = (1 - \text{ayah-set match}) + (1 - \text{word-assignment accuracy}) +$
 41 $(1 - \text{abstention accuracy})$. Detection compares the *set* of ayah ids, so a repeated ayah is a split
 42 concern rather than a detection error. Split accuracy is the fraction of gold transcript words placed in
 43 the correct ayah, compared after a metric-only orthographic canonicalization that folds hamza-bearing
 44 alif forms, alif maqsura and hamza seats and strips diacritics, so spelling noise is not scored as a
 45 segmentation error. An empty system scores 2.0 and the gold labels 0.0, and scoring the full set takes
 46 under a second, which matters for the agent track below.

Baseline	full set	held-out	held-out components	
Empty system	2.000	2.000	det 1.00, split 1.00	abstain 0
Incumbent application pipeline (hand-built)	0.785	0.760	det 0.17, split 0.09	abstain 0.50
Best of six coding-agent runs ¹	0.008	0.079	det 0.06, split 0.02	abstain 0

Table 1: Baselines on the released split. The incumbent is the segmentation stack of a deployed application, tuned over roughly two years.

47 The incumbent misses 17% of ayah sets and abstains on one of two held-out negatives. Six au-
 48 tonomous runs of two coding agents in a Karpathy-style fixed-metric loop all matched or beat it held
 49 out, the best by an order of magnitude.¹ That study is also why this competition keeps a hidden
 50 final test: without a held-out split the loop drove the score ten times lower largely by memorizing
 51 evaluation rows, hardcoding up to 41 ayah ids per run including literal transcript-to-answer lookups,
 52 and merely disclosing that a held-out set existed eliminated the literal memorization. A public score
 53 on a public split is not evidence of a better algorithm.

54 Residual challenges include cross-surah recitation, non-contiguous ayah sets and abstention; one
 55 missed held-out rejection costs half a point.

56 4 Competition plan, feasibility and ethics

57 The competition launches on the frozen benchmark with a public leaderboard on the released test split,
 58 and ranks on a hidden test set of recordings collected after the public release, labeled by the same
 59 protocol and published when the competition closes. A read-only production check has identified
 60 63 further recordings with 62 transcripts, forming 57 distinct text groups absent from the old export,
 61 as candidates for that collection; reuse permission, de-identification and overlap are checked first.
 62 These candidates are not yet a frozen test set; any group allocated to annotation practice or agent
 63 development is excluded from hidden final ranking. A submission is a single Python file, and scoring
 64 takes under a second, so an *agent track* for autonomous loops with per-experiment logs runs beside
 65 the human track under identical rules, with hardcoded-constant counts reported for every entry.

66 **Feasibility.** Harness, data, checksums and review sample are attached, and the baselines run offline
 67 without API keys or audio. **Ethics.** Text only, de-identified, adult users, disclosed review, CC BY 4.0;
 68 the task supports corrective feedback rather than surveillance, and abstention is scored precisely so a
 69 deployed system is rewarded for staying silent when it does not understand. **Participation.** Small
 70 data, sub-second evaluation, and a problem every Quran-learning application faces; Arabic NLP,
 71 speech and coding-agent groups are natural entrants. A companion submission covers the within-ayah
 72 mistake annotation built on the same recordings.

¹Autoresearch with coding agents: generalizers and metric-maximizers on Quran recitation data, AIST 2026.